MLOps Assignment 02 — Cats vs Dogs

MLOps Assignment 02 — Cats vs Dogs Image Classification Pipeline

Course: Machine Learning Operations (AIMLCZG523)
Institution: BITS Pilani
GitHub Repository: https://github.com/krupashankarsugi/mlops-assignment-2
Demo Video (< 5 min): https://drive.google.com/file/d/1tdaeuOKgOFs5fhL2cXUq0cGHA5jl5Azu/view?usp=share_link

---

1. Project Overview

This project implements an end-to-end MLOps pipeline for binary image classification (cat vs dog) in the context of a pet adoption platform. The pipeline covers the full ML lifecycle: dataset acquisition and versioning, exploratory analysis, preprocessing and augmentation, model training with experiment tracking, REST API serving, containerization, CI/CD automation with registry publishing, Kubernetes deployment gated by smoke tests, and post-deployment monitoring.

Objectives

- Version both source code and data reproducibly (Git + DVC)
- Train a baseline CNN and track every experiment with MLflow
- Package the model behind a REST API and containerize it
- Automate testing, image build and registry publishing with GitHub Actions
- Deploy the container automatically and gate the pipeline on smoke tests
- Monitor request volume, latency and post-deployment model accuracy

Technology Stack

Concern	Tool
Language	Python 3.10
Deep learning	PyTorch 2.2.2, torchvision 0.17.2
Experiment tracking	MLflow 2.14.1
Data versioning	DVC 3.51.2
API	FastAPI 0.111.0 + Uvicorn
Testing	pytest 8.2.2, ruff
Containerization	Docker (multi-stage)
Registry	GitHub Container Registry (GHCR)
CI/CD	GitHub Actions
Orchestration	Kubernetes (k3d in CI, minikube locally)
Monitoring	Prometheus + in-app counters
Version control	Git, GitHub

2. Dataset

Source: Cats and Dogs binary classification dataset (Kaggle)
URL: https://www.kaggle.com/datasets/bhavikjikadara/dog-and-cat-classification-dataset

Size: 24,998 images — 12,499 cat, 12,499 dog (perfectly balanced)

This is the dataset linked from the assignment brief. `src/data/download.py` fetches it from Kaggle's public dataset endpoint, which serves this dataset **without requiring credentials**, so `dvc repro` reproduces the pipeline on a clean machine. The Kaggle CLI is used instead when `~/.kaggle/kaggle.json` is configured.

2.1 Source Verification

An earlier iteration of this project sourced the images from Microsoft's release of the Cats vs Dogs corpus (`kagglecatsanddogs_5340.zip`) rather than from Kaggle. The two were compared directly before standardising on the Kaggle dataset:

Check	Result
Archive layout	Identical — `PetImages/Cat`, `PetImages/Dog`
Files in Microsoft, absent from Kaggle	Exactly 2: `Cat/666.jpg`, `Dog/11702.jpg`
Files in Kaggle, absent from Microsoft	0
Sampled images byte-compared (SHA-256)	6/6 identical
Processed training data (4,000 images)	**Byte-identical between both sources**

The two files Kaggle omits are the corpus's well-known corrupt JPEGs, which `is_valid_image()` rejects regardless of source. Because preprocessing selects the first *valid* images per class, both sources yield the

same 4,000 processed images — verified by hashing the entire processed tree from each. **The trained model and every metric in this report are therefore unaffected by the source swap.**

2.2 Working Subset

`params.yaml` caps training at 2,000 images per class so the full pipeline runs end-to-end on a laptop in minutes. Setting `data.max_images_per_class: 0` uses all 24,998 images.

Split	Cat	Dog	Total
Train	1,600	1,600	3,200
Validation	200	200	400
Test	200	200	400
Total	**2,000**	**2,000**	**4,000**

3. Exploratory Data Analysis (EDA)

Notebook: `notebooks/01_eda.ipynb` (executed, with outputs saved)

3.1 Class Balance

The corpus is exactly balanced at 12,499 images per class. Accuracy is therefore a fair headline metric and no class weighting is required.

3.2 Image Dimensions

Sampled across both classes:

Property	Min	Median	Max
Width (px)	51	438	500
Height (px)	72	375	500
Aspect ratio	0.39	1.23	2.75

Images vary widely in both size and aspect ratio. This is the direct justification for resizing everything to a fixed **224×224 RGB** tensor — the standard input for the CNN architectures used here.

3.3 Corrupt Files

The Cats vs Dogs corpus is known to contain zero-byte and truncated JPEGs that crash training data loaders. The Kaggle upload has already removed the two worst offenders (`Cat/666.jpg` , `Dog/11702.jpg`), and `is_valid_image()` still screens every file before the split, so the pipeline is safe against either source.

3.4 Pixel Intensity

The two classes have near-identical pixel intensity distributions, so there is no trivial colour or brightness shortcut. The model must learn genuine shape and texture features, which is why a from-scratch CNN needs several epochs before validation accuracy moves off chance level.

3.5 Findings That Shaped the Pipeline

Observation	Pipeline response
Perfectly balanced classes	Plain accuracy is a fair metric; no class weighting
Corrupt / zero-byte JPEGs present	`is_valid_image()` filters before the split
Highly variable dimensions	Fixed 224×224 RGB resize
No colour/intensity shortcut	Augmentation and sufficient epochs genuinely needed
Splits must be reproducible	Seeded `split_indices()`, verified by unit tests

---

4. Data Preprocessing & Versioning

4.1 Preprocessing

`src/data/preprocess.py` converts every image to 224×224 RGB JPEG and splits **80 / 10 / 10** into train / validation / test with a fixed seed (42). Each class is split independently, so every split stays balanced.

4.2 Augmentation

Applied to the **training split only** (`src/data/dataset.py`):

- Random resized crop (scale 0.8–1.0)
- Random horizontal flip (p = 0.5)
- Random rotation (±15°)
- Colour jitter (brightness / contrast / saturation 0.2)
- Normalisation with ImageNet statistics

Validation, test **and the production API** all share a single deterministic `build_eval_transform()`. Reusing the same function on the serving path is what prevents preprocessing from drifting between training and production.

4.3 DVC Pipeline

`dvc.yaml` declares three stages:

```
download ⟶ preprocess ⟶ train
```

`dvc repro` re-runs only the stages whose dependencies or `params.yaml` values changed. `data/raw/`, `data/processed/` and `models/model.pt` are DVC-tracked outputs kept out of Git; source code and configuration are versioned in Git.

```
dvc repro            # reproduce the full pipeline
dvc dag              # visualise stage dependencies
dvc metrics show     # test metrics from reports/metrics.json
```

---

5. Model Development & Experiment Tracking

5.1 Models Trained

SimpleCNN (baseline) — `src/models/cnn.py` - Four Conv-BN-ReLU-MaxPool blocks (3 → 32 → 64 → 128 → 256) - Global average pooling → dropout → FC(256→128) → FC(128→2) - 422,050 trainable parameters, trained from scratch - Adam, lr = 1e-3, weight decay = 1e-4, batch size 32, 8 epochs

ResNet18Transfer (comparison) — `src/models/cnn.py` - ImageNet-pretrained ResNet-18, backbone frozen, fresh 2-way head - 11,177,538 total parameters, 1,026 trainable - Same optimiser settings, 5 epochs

5.2 Evaluation Metrics (test split, 400 images)

Metric	SimpleCNN (baseline)	ResNet-18 (transfer)
Accuracy	74.75%	**97.50%**
Precision	70.37%	97.50%
Recall	85.50%	97.50%
F1-Score	77.20%	97.50%
ROC-AUC	0.841	**0.998**
Epochs	8	5

Confusion matrices (test split):

	Baseline pred. cat	pred. dog	Transfer pred. cat	pred. dog
true cat	128	72	195	5
true dog	29	171	5	195

The baseline satisfies the assignment's requirement for a from-scratch CNN and comfortably beats the logistic-regression-on-flattened-pixels alternative the brief offers. It is honest about its limits: 74.75% from ~422k parameters on 3,200 images in 8 epochs, with recall (85.5%) noticeably higher than precision (70.4%) — it over-predicts "dog".

The transfer run exists to give MLflow a second run to compare and to quantify what a pretrained feature extractor buys on identical data. **The transfer model is the one deployed.**

5.3 MLflow Experiment Tracking

MLflow tracked all experiments with a local file backend (`mlruns/`).

Logged per run: - Parameters: architecture, epochs, batch size, learning rate, weight decay, dropout, image size, seed, optimiser, parameter counts, split sizes, augmentation summary - Per-epoch metrics: train/validation loss and accuracy, learning rate - Test metrics: accuracy, precision, recall, F1, ROC-AUC - Artifacts: confusion matrix (PNG), training curves (PNG), serialized model - Model registry entry: `cats-vs-dogs-cnn`

Experiment name: `cats-vs-dogs`
MLflow UI: `make mlflow-ui` → http://localhost:5000

5.4 Model Promotion

Experiment tracking is only useful if the winning run can actually ship. `scripts/promote_model.py` ranks the runs in the MLflow store by a chosen metric, pulls the best run's weights and rewrites `models/model.pt` plus the metadata sidecar the API reads:

```
python scripts/promote_model.py --best-by test_accuracy
```

This is how the 97.5% ResNet-18 run became the served model.

---

6. Model Packaging & API Serving

6.1 Model Serialization

The checkpoint (`models/model.pt`, PyTorch `.pt`) stores the weights **plus** the class names, image size and normalisation constants. The serving side never has to guess how the model was trained.

6.2 API Endpoints

Base URL: `http://localhost:8000`

Method	Endpoint	Purpose
`GET`	`/health`	Liveness + whether the model deserialised
`GET`	`/ready`	Readiness — 503 until the model is usable
`POST`	`/predict`	Single image → label + class probabilities
`POST`	`/predict/batch`	Up to 16 images per call
`GET`	`/model-info`	Served checkpoint metadata + test metrics
`GET`	`/stats`	In-app counters and latency percentiles
`GET`	`/metrics`	Prometheus exposition format
`GET`	`/docs`	Interactive OpenAPI documentation

6.3 Example Request

```
curl -F "file=@data/processed/test/dog/dog_10042.jpg" http://localhost:8000/predict
```

```
{
  "label": "dog",
  "confidence": 0.999175,
  "probabilities": {"cat": 0.000825, "dog": 0.999175},
  "inference_ms": 71.129,
  "request_id": "caab5cc3dd1d"
}
```

6.4 Environment Specification

All key ML libraries are pinned to exact versions:

- `requirements.txt` — runtime dependencies shipped in the image
- `requirements-train.txt` — adds MLflow, DVC, scikit-learn, matplotlib
- `requirements-dev.txt` — adds pytest, ruff, httpx

Two pins exist for documented reasons: `setuptools==69.5.1` (MLflow 2.14 still imports `pkg_resources`) and `pathspec==0.12.1` (DVC's own `pathspec>=0.10.3` is too loose — pathspec 1.x removed a private API DVC imports).

---

7. Testing

56 tests, all passing, run via pytest.

File	Tests	Covers
`tests/test_preprocess.py`	17	`resize_image`, `split_indices`, `is_valid_image` — ratios, determinism, disjointness, corrupt-file rejection
`tests/test_inference.py`	22	Model factory, preprocessing, prediction shape/probabilities, metric computation
`tests/test_api.py`	12	Health, readiness, predict, batch, error paths, `/stats`, `/metrics`

The assignment requires a unit test for at least one data preprocessing function and one model utility/inference function; both categories are covered several times over.

Tests build a synthetic model and synthetic images in fixtures, so CI never needs the 800 MB dataset or a trained checkpoint.

```
make test     # pytest with coverage
```

---

8. Containerization

8.1 Dockerfile

Multi-stage build: stage 1 resolves pinned dependencies into a virtualenv, stage 2 copies only that venv plus application code, so build toolchains never ship.

Design choices: - **CPU-only torch wheels** (`download.pytorch.org/whl/cpu`) — roughly 4× smaller than the default CUDA build - Runs as **non-root** (uid 10001), read-only root filesystem, all Linux capabilities dropped - `HEALTHCHECK` wired to `/health`

Final image: 218 MB compressed

8.2 Local Verification

```
docker build -t cats-vs-dogs-api:latest .
docker run -d -p 8000:8000 cats-vs-dogs-api:latest
curl -F "file=@test.jpg" http://localhost:8000/predict
```

The container was additionally verified under `docker run --read-only --user 10001`, reproducing the Kubernetes `securityContext` exactly — model loads, `/ready` returns 200, predictions serve.

8.3 Docker Compose

`docker-compose.yml` brings up the API plus Prometheus in one command as an alternative deployment target:

```
make compose-up
```

---

9. CI/CD Pipeline (GitHub Actions)

9.1 CI — `.github/workflows/ci.yml`

Triggered on every push and pull request to `main`.

```
┌──────────────────────────────────────────┐
│  Job 1: Lint & Unit Tests                │
│    checkout → setup Python 3.10          │
│    → install pinned deps → ruff          │
│    → pytest (56 tests) → coverage        │
└──────────────────┬───────────────────────┘
                   ▼
┌──────────────────────────────────────────┐
│  Job 2: Build & Push Image               │
│    restore model (DVC or CI fallback)    │
│    → login GHCR → buildx build           │
│    → push ghcr.io/<owner>/<repo>         │
│    → run image + assert a prediction     │
└──────────────────────────────────────────┘
```

Image tags: `latest`, branch name, and `sha-<short>`. Pull requests build the image but do not push it.

The final step is deliberate: CI starts the image it just built and asserts it serves a real prediction. An image that compiles but cannot infer fails in CI rather than in CD.

Artifact publishing: images are pushed to GitHub Container Registry at `ghcr.io/krupashankarsugi/mlops-assignment-2`.

9.2 CD — `.github/workflows/cd.yml`

Triggered when CI succeeds on `main`.

1. Provision a **k3d Kubernetes cluster** on the runner
2. **Pull** the new image from GHCR and import it into the cluster
3. **Apply** the manifests and `kubectl set image` to the new tag
4. **Wait** for the rollout (`kubectl rollout status`)
5. **Smoke test** — `/health` , `/ready` and one real `/predict`
6. **Roll back** automatically (`kubectl rollout undo`) if the smoke test fails
7. Tear down the cluster

The smoke test exits non-zero on any failure, which fails the pipeline.

9.3 Verified Results

Both workflows pass on GitHub-hosted runners:

```
CI  #2  completed  success
CD  #2  completed  success
```

CD output:

```
configmap/cats-vs-dogs-config created
deployment.apps/cats-vs-dogs-api created
service/cats-vs-dogs-api created
deployment "cats-vs-dogs-api" successfully rolled out
deployment.apps/cats-vs-dogs-api   2/2   2 available

=== Smoke test against http://localhost:8000 ===
[2/4] /health ok      [3/4] /ready ok      [4/4] /predict ok
=== SMOKE TEST PASSED ===
```

---

10. Kubernetes Deployment

10.1 Manifests

File	Resource
`k8s/deployment.yaml`	Deployment — 2 replicas
`k8s/service.yaml`	Service — NodePort 30080
`k8s/configmap.yaml`	ConfigMap — runtime configuration
`k8s/hpa.yaml`	HorizontalPodAutoscaler (optional)

Deployment configuration: - Rolling update with `maxUnavailable: 0` for zero-downtime releases - `startupProbe` (30 × 5s), `livenessProbe` on `/health` , `readinessProbe` on `/ready` - Resource requests

250m CPU / 512Mi, limits 1000m CPU / 1536Mi - `runAsNonRoot`, `readOnlyRootFilesystem`, all capabilities dropped - Prometheus scrape annotations

The `startupProbe` matters: PyTorch takes several seconds to import and deserialise the checkpoint, and without it the liveness probe would kill the pod mid-cold-start.

10.2 Deployment Verification

The rollout is verified on **two** independent clusters.

CD pipeline (k3d, GitHub-hosted runner) — see Section 9.3: 2/2 replicas Ready, rollout succeeded, smoke test passed.

Local minikube via `scripts/deploy_local.sh` (build → load → apply → wait → smoke test):

```
deployment.apps/cats-vs-dogs-api   2/2   2 up-to-date   2 available
service/cats-vs-dogs-api           NodePort   10.107.207.87   80:30080/TCP
configmap/cats-vs-dogs-config      4 keys
pod/cats-vs-dogs-api-f44997966-rkrwn   1/1   Running
pod/cats-vs-dogs-api-f44997966-vgswt   1/1   Running

=== Smoke test ===
[2/4] /health ok      [3/4] /ready ok
[4/4] /predict ok  (label=dog, confidence=0.9873, latency=3629.9ms)
=== SMOKE TEST PASSED ===
```

The local run is the stronger check of the two: it serves the **trained** checkpoint, so the prediction above is a genuine 98.7%-confidence classification of a held-out test image, whereas CD deploys the untrained CI placeholder and can only prove the deployment path.

The first cold-start `/predict` takes ~3.6 s because PyTorch lazily initialises on the first forward pass; steady-state latency is ~33 ms (Section 11.3).

10.3 A Bug the Security Context Caught

`readOnlyRootFilesystem: true` exposed a genuine defect. Rebuilding the transfer architecture to load a checkpoint was calling `torchvision.resnet18(weights=IMAGENET1K_V1)`, which downloads ImageNet weights into `~/.cache/torch` — weights immediately overwritten by the checkpoint's own `state_dict`.

Under Docker Compose this merely wasted a download on every container start. Under a read-only root filesystem the pod never became ready. The fix is `build_model(..., pretrained=False)` on the load path, which also removes a network dependency at startup. `tests/test_inference.py` guards the regression.

---

11. Monitoring & Logging

11.1 Request/Response Logging

Structured, one JSON object per line, so logs are greppable in Docker and Kubernetes:

```
{"ts":"2026-08-24T13:37:34Z","level":"INFO","service":"cats-vs-dogs-api",
 "message":"prediction served","request_id":"25f11a66a0b0","event":"prediction",
 "filename":"cat_11022.jpg","bytes":9940,"label":"cat","confidence":0.5339,
 "inference_ms":13.61}
```


No image bytes are ever logged — only the filename, payload size and outcome. Every request receives an `x-request-id` (honoured from the inbound header when present) echoed back in the response for tracing.

11.2 Metrics

Two complementary surfaces:

`/stats` — in-app counters: total requests, predictions, errors, prediction mix per class, mean and p95 latency.

`/metrics` — Prometheus: - `inference_requests_total{endpoint,method,status}` - `predictions_total{predicted_class}` - `inference_errors_total{endpoint,reason}` - `inference_request_latency_seconds` (histogram)

Verified live — Prometheus scraped the containerized API successfully (target UP) and PromQL `histogram_quantile` returned p95 latency.

11.3 Post-Deployment Model Performance Tracking

`scripts/monitor_performance.py` replays a batch of held-out test images — whose true labels are known from the directory name — against the **deployed** service and compares served predictions with ground truth. This measures the model *as deployed* (container + preprocessing + weights), not as trained.

Results over 60 labelled requests:

Metric	Value
Accuracy	95.0% (57/60)
Mean confidence	0.964
Latency p50	32.9 ms
Latency p95	62.9 ms
Failed requests	0

Class	Precision	Recall	F1	Support
cat	0.909	1.000	0.952	30
dog	1.000	0.900	0.947	30

Outputs are written to `reports/monitoring/` — `performance_report.json`, a `predictions.jsonl` audit trail, and a live confusion matrix.

12. Project Structure

```
.
├── api/                        FastAPI inference service
│   ├── main.py                 endpoints + observability middleware
│   ├── monitoring.py           JSON logging, in-app + Prometheus metrics
│   └── schemas.py              request/response models
├── src/
│   ├── config.py               typed access to params.yaml
│   ├── data/                   download, preprocess, datasets/augmentation
│   └── models/                 CNN definitions, training, prediction
├── tests/                      56 pytest tests
├── scripts/                    smoke test, monitoring, promotion, packaging
├── k8s/                        Deployment, Service, ConfigMap, HPA
├── monitoring/                 Prometheus configuration
├── notebooks/01_eda.ipynb      executed exploratory analysis
├── .github/workflows/          ci.yml, cd.yml
├── reports/                    metrics, figures, monitoring reports
├── dvc.yaml / dvc.lock         data → model pipeline
├── params.yaml                 all tunable parameters
├── Dockerfile                  multi-stage inference image
├── docker-compose.yml          API + Prometheus stack
└── Makefile                    task shortcuts
```

13. Setup Instructions

```
# 1. Clone repository
git clone https://github.com/krupashankarsugi/mlops-assignment-2.git
cd mlops-assignment-2

# 2. Environment (Python 3.10)
make setup

# 3. Data — download ~800 MB corpus, resize and split
make data

# 4. Train the baseline CNN (logs to MLflow, writes models/model.pt)
make train

# 5. Tests
make test

# 6. Serve locally
make serve                 # http://localhost:8000/docs

# 7. Container
make docker-build
make docker-run
make smoke                 # post-deploy smoke test

# 8. Kubernetes (local minikube)
./scripts/deploy_local.sh

# 9. Post-deployment monitoring
make monitor
```

Run `make help` to list every target. The whole pipeline can also be reproduced with `dvc repro`.

Example prediction:

```
curl -F "file=@data/processed/test/cat/cat_10017.jpg" \
     http://localhost:8000/predict
```

14. Key Findings & Conclusions

Transfer learning dominates a from-scratch CNN at this data scale. The baseline reached 74.75% test accuracy from 422k parameters over 8 epochs; a frozen ImageNet ResNet-18 with only 1,026 trainable parameters reached 97.5% in 5 epochs. With 3,200 training images, pretrained features are worth far more than a bespoke architecture.

Sharing the preprocessing function between training and serving eliminates a whole class of bug. The API calls the same `build_eval_transform()` used for validation and test, so serving-time preprocessing cannot silently drift from evaluation-time preprocessing.

Hardening the container found a real defect. `readOnlyRootFilesystem` exposed an unnecessary ImageNet weight download on the model-load path that Docker Compose had been silently tolerating (Section 10.3). Security constraints acted as a correctness test.

Verifying the artifact in CI is worth more than verifying the code. The CI step that runs the built image and asserts a prediction catches failures that unit tests structurally cannot — missing runtime libraries, a broken checkpoint, a bad entrypoint.

Post-deployment measurement is not the same as test-set measurement. The deployed model scored 95.0% on 60 replayed labelled requests against a 97.5% test accuracy, while also surfacing latency (p50 32.9 ms, p95 62.9 ms) that offline evaluation never reports.

Limitations

- Training used 4,000 of the available 24,998 images to keep the pipeline laptop-runnable; `data.max_images_per_class: 0` lifts this.
- No DVC remote is configured, so CI falls back to a placeholder checkpoint when building images. The pod deployed by CD therefore carries untrained weights — CD verifies the deployment path, not model quality.
- Training used a 4,000-image subset, so the reported accuracy is not what the full 24,998-image corpus would yield.

---

15. Repository

GitHub: https://github.com/krupashankarsugi/mlops-assignment-2

Container image: `ghcr.io/krupashankarsugi/mlops-assignment-2:latest`

The trained checkpoint (`models/model.pt` , 43 MB) is committed to the repository, so a fresh clone can serve predictions without retraining:

```
git clone https://github.com/krupashankarsugi/mlops-assignment-2.git
cd mlops-assignment-2 && make setup && make serve
```

CI/CD: https://github.com/krupashankarsugi/mlops-assignment-2/actions

Demo Video

A screen recording (4 min 49 s) demonstrating the complete MLOps workflow from a code change through CI, image publication and automated deployment to a prediction served by the deployed model:

https://drive.google.com/file/d/1tdaeuOKgOFs5fhL2cXUq0cGHA5jl5Azu/view?usp=share_link

---